# Patrones estructurales

Hoja de ejercicios (Alumno)

## Hoja de ejercicios: Patrones estructurales, *Composite*

### Patrón *Composite*

#### Escenario

Se está desarrollando una aplicación para gestionar una flota espacial. Dentro del sistema aparecen dos tipos de elementos:

- **Naves individuales**, representadas por la clase `Ship`.
- **Flotas o escuadrones**, representadas por la clase `Fleet`.

Tanto una nave individual como una flota deben poder someterse a las mismas operaciones generales del dominio, por ejemplo:

- mostrar sus datos,
- calcular su consumo total de combustible.

#### Objetivo del ejercicio

El objetivo del ejercicio es conseguir un **tratamiento uniforme** de todos los elementos de la estructura, de forma que una nave individual y una flota puedan manipularse mediante el tratamiento más uniforme posible.

#### Proyecto proporcionado (versión sin patrón)

En el fichero `Composite_NoPattern.zip` se proporciona un proyecto Java (en el paquete `space`) con una primera solución **sin aplicar el patrón *Composite***.

En esta versión:

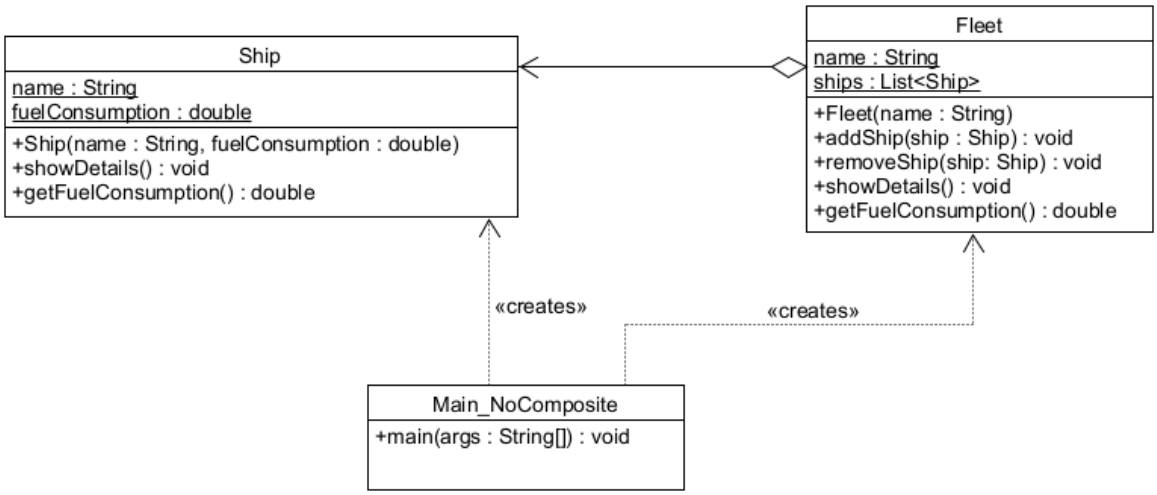
- `Ship` y `Fleet` son clases distintas que no pertenecen a la misma jerarquía.
- `Fleet` sí que recoge la naturaleza de estar compuesta por un conjunto de objetos de tipo `Ship`.

Analiza que en el código del programa principal, al intentar definir un tratamiento uniforme para todos los objetos involucrados (`Ship`s y `Fleet`) es necesario recurrir a tratamiento condicional basado en el uso de técnicas como `instance of`. De otro modo habría que realizar algo similar en el programa principal. Observa que tienes ambas opciones en el método `main`:

- Tratamiento explícito y separado para objetos de tipo `Ship` y objetos de tipo `Fleet`
- Tratamiento unificado que conlleva una comprobación y tratamiento condicional basándose en técnicas como `instance of` para saber qué es lo que hay que hacer.

### UML de la versión sin patrón

A continuación se muestra el diagrama UML de la solución sin patrón.




### Qué es lo que funciona mal

Aunque la versión proporcionada funciona, presenta varios problemas estructurales:

- La lógica de la aplicación debe saber si está trabajando con una `Ship` o con una `Fleet`.
- No existe un tipo común que permita tratar ambos elementos de forma uniforme.
- El uso de comprobaciones de tipo (`instanceof`) es una señal clara de que falta una abstracción adecuada.
- La clase `Fleet` solo puede contener naves y no puede contener otras flotas, por lo que no se puede modelar una estructura jerárquica real.

### Se pide

Debes desarrollar una nueva versión del sistema aplicando el patrón estructural *Composite*. La solución con patrón deberá incorporar, al menos, los siguientes elementos:

- Un **Component** común:
  - `SpaceUnit`
- Una **Leaf**:
  - `Ship`
- Un **Composite**:
  - `Fleet`

### Métodos obligatorios

La abstracción común `SpaceUnit` deberá declarar, al menos, los siguientes métodos:

- `public void showDetails();`
- `public double getFuelConsumption();`

La clase `Fleet` deberá poder contener elementos del tipo abstracto `SpaceUnit`, y no solo naves individuales.

Además, en la clase `Fleet` deberán existir operaciones para gestionar sus elementos hijos, como por ejemplo:

- `public void add(SpaceUnit unit);`
- `public void remove(SpaceUnit unit);`

### Qué debe cambiar respecto a la versión sin patrón

En la nueva versión:

- la aplicación principal no debe distinguir entre naves y flotas mediante comprobaciones de tipo;
- debe ser posible aplicar las mismas operaciones a una nave o a una flota a través del tipo común `SpaceUnit`;
- una flota debe poder contener tanto naves individuales como otras flotas.

### Programa principal esperado

Tu solución debe permitir escribir una lógica de aplicación principal como la siguiente (o equivalente), donde una nave y una flota se manipulan del mismo modo:

```
SpaceUnit unit1 = new Ship("Falcon-1", 10.0);
SpaceUnit unit2 = new Fleet("Main Fleet");
unit1.showDetails();
unit2.showDetails();
System.out.println(unit1.getFuelConsumption());
System.out.println(unit2.getFuelConsumption());
```

```java
SpaceUnit ship1 = new Ship("Falcon-1", 10.0);
SpaceUnit ship2 = new Ship("Falcon-2", 12.0);
SpaceUnit ship3 = new Ship("Command-1", 20.0);

Fleet squadron = new Fleet("Alpha Squadron");
squadron.add(ship1);
squadron.add(ship2);

Fleet mainFleet = new Fleet("Main Fleet");
mainFleet.add(squadron);
```

```
        mainFleet.add(ship3);

        System.out.println("=== WITH Composite ===");

        processUnit(ship1);
        System.out.println();

        processUnit(mainFleet);
```

### Objetivo final

El objetivo principal del ejercicio es mostrar cómo el patrón *Composite* permite tratar de forma uniforme objetos simples y objetos compuestos dentro de una misma estructura jerárquica.